

Tecnológico de Monterrey
Escuela de Ingeniería y Ciencias

Campus Santa Fe

Analysis and Design of Advanced Algorithms

TC2038.622

Professor: Lizbeth Peralta Malváez

“Integrated Activity 1”

Group: 601

Team:

Daniela Angulo Solís	A01028153
Jonathan Uriel Anzures García	A01277273
Josué Gómez Arteaga	A01787212

Due date:

October 9th, 2026

Integrated Activity 1

Algorithms applied.

For the first part of the activity, where we have to check if the malicious code is in the transmission, we use the KMP algorithm because we can use the mcode as the pattern we have to search for in the transmission. Knuth-Morris-Pratt (KMP) is a string-matching algorithm developed by Donald Knuth, Vaughan Pratt, and James H. Morris in 1977 (Geeks for Geeks, 2025). The KMP algorithm searches for a pattern inside a text in O(n+m) time, where n is the text length and m is the pattern length. It avoids rechecking characters by precomputing the pattern into an LPS array. The KMP algorithm works in two main steps:

1. Preprocessing the Pattern: Before searching, KMP creates an array (LPS array) based on the pattern that helps determine how much of the pattern has already been matched when a mismatch occurs.

Index(i)	0	1	2	3	4	5	6	7	8
Pattern	A	B	A	B	C	A	B	A	B
LPS Value	0	0	1	2	0	1	2	3	4

Img 1. Example of an LPS array (Geeks for Geeks, 2025)

2. Searching the Text: Using the LPS array, KMP quickly skips over parts of the text where it's certain the pattern can't match, making the search more efficient.

For the second part of the activity, where we have to find the longest palindrome in each transmission, we use Manacher's Algorithm, which is an algorithm designed to find palindromic substrings of a string in linear time (GeeksforGeeks, 2025). Other approaches, like brute force, dynamic programming, or center expansion, take $O(n^3)$ or $O(n^2)$ time, and hashing combined with binary search takes O(n log n), which is too slow for the large transmission files the program will be tested with (GeeksforGeeks, 2025). Manacher's Algorithm finds the longest palindromic substring in O(n) time, where n is the length of the transmission; each character of the

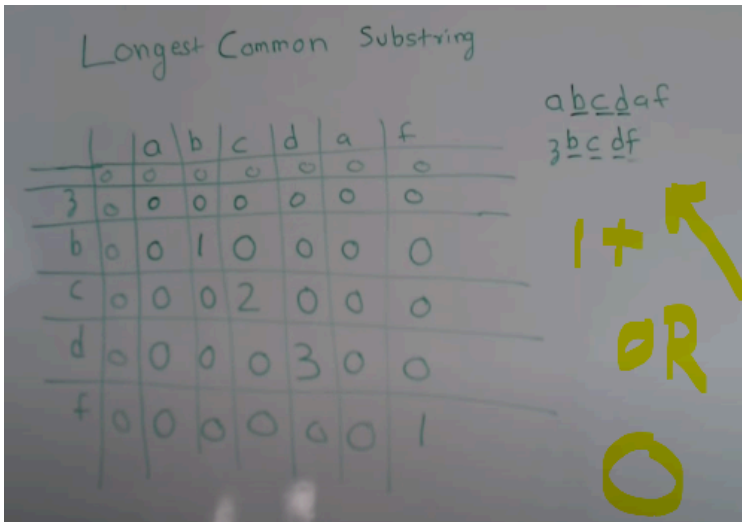
transformed string is visited at most once during expansion, and the mirror values prevent redundant checks. The algorithm works in three main steps:

1. Preprocessing the String: Palindromes can have an odd length (like "ABA") or an even length (like "ABBA"), and handling both cases separately makes the code more complex. To treat them the same way, the algorithm inserts a "#" between each character, so every palindrome becomes odd-length, and both the characters and the spaces between them can work as centers. (GeeksforGeeks, 2025).
2. Computing the Radius Array: The algorithm goes through the transformed string once and fills an array p[], where p[i] stores the radius (number of characters on one side) of the longest palindrome centered at position i. It also keeps track of the boundaries of the rightmost palindrome found so far. If the current position is inside that palindrome, it uses the value of its mirror position (on the other side of the center) as a starting point, taking the minimum between that value and the distance to the right boundary, so it doesn't assume anything beyond what it already knows (GeeksforGeeks, 2025).
3. Expanding and Updating: From that starting point, the algorithm compares the characters on both sides of the center and keeps expanding while they match. If the new palindrome reaches beyond the current right boundary, it becomes the new rightmost palindrome (GeeksforGeeks, 2025). Finally, the largest value in p[] gives the length of the longest palindrome, which we convert back into its start and end positions (starting at 1) in the original transmission.

For the final part, where we have to analyze how similar the transmissions are, we look for the longest common substring between both files using Dynamic Programming. Dynamic Programming is an algorithmic technique that optimizes plain recursion by storing the results of subproblems, so they do not have to be computed again when they are needed later, which usually reduces the time complexity from exponential to polynomial (GeeksforGeeks, 2026). A problem can be solved with Dynamic Programming when it has overlapping subproblems, and most classic problems also have optimal substructure. The time complexity of the algorithm is $O(n \times m)$, where n and m are the lengths of the transmissions, because every character of one transmission is compared with every character of the other one.

The longest common substring is a variation of the Longest Common Subsequence (LCS) problem. The difference is that in a subsequence the characters can be separated, as long as they keep their relative order (GeeksforGeeks, 2026), while in a substring they must be contiguous. Because of this, the only change in the solution is what happens when two characters do not match: instead of keeping a previous value, the length goes back to 0 (Vidhu, 2021.). The algorithm works in three main steps:

1. Defining the State: Following the steps to solve a Dynamic Programming problem, we first decide the state, which is the smallest set of parameters that uniquely identifies a subproblem (GeeksforGeeks, 2024). We use a table dp[i][j] that stores the length of the longest common substring that ends exactly at character i of transmission1 and at character j of transmission2. The first row and the first column are filled with 0, since the common substring between any string and an empty string has a length of 0 (Vidhu, n.d.).
2. Filling the Table: For each pair of characters, if they are equal, the common substring that ended at the previous characters grows by one, so dp[i][j] = 1 + dp[i-1][j-1]. If they are different, the common substring breaks, and dp[i][j] = 0 (Vidhu, n.d.).



Img 2. Example of the DP table for the longest common substring (Vidhu, n.d.)

3. Finding the Longest Substring: Since a common substring can end anywhere in the table, not necessarily in the last cell, the answer is the largest value in the table. While filling it, we keep track of the maximum length and the index in transmission1 where it ends

(Vidhu, n.d.). With these two values, we calculate the start and end positions (starting at 1) of the longest common substring in transmission1.

Results analysis.

The program was executed with the five test files (transmission1.txt, transmission2.txt, mcode1.txt, mcode2.txt and mcode3.txt). Line breaks were removed when reading the files, so every position refers to the characters of the transmission as one continuous string, starting at 1.

Part 1. Malicious code in the transmissions: The code in mcode1.txt (C3ABBA) was found in both transmissions, and in both cases, it starts at position 6. The codes in mcode2.txt (C3ABAB) and mcode3.txt (C3ABBBABABABB) were not found in either transmission, so the program displays false for them. Similar sequences appear in the files, such as 3BBABABABB, but the complete code never appears, which shows that the search only reports an exact match of the whole sequence.

```
--- Part 1. mcode in transmissions: ---
true 6
false
false
true 6
false
false
```

Img 3. Screenshot of Part 1

Part 2. Longest palindrome: In transmission1.txt, the longest mirrored code starts at position 188 and ends at position 204 (ABBABBABABABBABBA). In transmission2.txt, it starts at position 31 and ends at position 39 (BBABABABB). Because Manacher's algorithm inserts a special character between each character, it finds both odd- and even-length palindromes in a single pass.

```
--- Part 2. longest palindrome in both transmissions: ---
188 204
31 39
```

Img 4. Screenshot of Part 2

Part 3. Longest common substring: The longest common substring starts at position 1 and ends at position 164 of transmission1.txt. Transmission2.txt has exactly 164 characters, which means its entire content is identical to the first 164 characters of transmission1.txt.

```
--- Part 3. longest substring between transmissions: ---
1 164
```

Img 5. Screenshot of Part 3

Work division.

Team member	Work done
Daniela Angulo	Description of algorithms applied, code “Longest Palindrome” header and “Longest Substring” header, plus txt files
Josué Gómez	Results analysis, code “Longest Palindrome” header
Uriel Anzures	Code Main, “Longest Substring” header, and “Malicious Code” header

References.

Geeks for Geeks. (2025, 23 julio). *Knuth-Morris-Pratt in C++*. Geeks For Geeks. https://www.geeksforgeeks.org/cpp/knuth-morris-pratt-in-c/

Prefix function - Knuth-Morris-Pratt - Algorithms for Competitive Programming. (s. f.). https://cp-algorithms.com/string/prefix-function.html

GeeksforGeeks. (2025, 30 julio). *Manacher's Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/manachers-algorithm-linear-time-longest-palindromic-substring-part-1/

GeeksforGeeks. (2026, 11 septiembre). *Longest common subsequence (LCS)*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/longest-common-subsequence-dp-4/

GeeksforGeeks. (2024, 23 diciembre). *Steps to solve a Dynamic Programming Problem*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/solve-dynamic-programming-problem/

GeeksforGeeks. (2026b, septiembre 17). *Dynamic Programming (DP) Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/introduction-to-dynamic-programming-data-structures-and-algorithm-tutorials/

Vidhu. (2021, 15 junio). *Longest common substring*. LeetCode. https://leetcode.com/discuss/post/1273766/longest-common-substring-by-vidhuv9-wz54/